

The One Arm AI Slot Machine

Experiment

Team: 13, Hardcoders · Model: Gemini-3-flash-preview ·
CSE110 SP26

Experiment Design



Frozen Prompt

**Identical
every run**

original-prompt.txt



Clean Sessions

50 runs

No memory / no edits



Model Fixed

Gemini-3-flash-Preview

Same params always



Refinement

4 rounds

One shot per round

50 → Baseline

5 → Round 1

3 → Round 2

2 → Round 3

1 → **Final**

Baseline Results — 50 Runs

100 %

Ran in browser

46 %

Broken / errors

[6,256.5]

Avg output tokens

[689s]

Avg wall-clock time

Quality Distribution

✓ Fully functional + polished 27 54%

⚠ Functional but rough / unthemed 4 8%

✗ Broken / did not run 19 38%

Key Drift Observations

- › Structural: all 3-file split
- › Theme: Token + Spins + diverse theme
- › Features: Sound, leaderboard, animations (not all)

Refinement Journey

Round 1

50 → 5

candidates

Candidates:
004, 022, 026, 028, 037

Round 2

5 → 3

candidates

Prompt:
refinement-prompt-step2.txt

Candidate:
004, 022, 028

Round 3

3 → 2

candidates

Prompt:
refinement-prompt-step3.txt

Candidate:
004, 026

Final

2 → 1

candidates

Prompt:
refinement-prompt-step4.txt

Candidate:
026

Every refinement was a single, clean session — no back-and-forth, no hand-editing.

Key Findings



High Variation, Same Prompt

Across all 50 runs, significant variation was observed in both performance and output. Wall-clock time, animation smoothness, and token usage despite identical prompts. The generated apps also differed substantially in theme interpretation, available features, gameplay, and overall speed, demonstrating that the same prompt can yield meaningfully different results each time.



What Refinement Bought Us

Refinement prompts produced noticeable improvements in UI, the game models, and satirical tone across rounds. However, gains were inconsistent, results still varied depending on how the model interpreted each refinement prompt, and some candidates remained non-functional even after refinement.



Where Prompting Hit a Ceiling

Some models interpreted the concept of "spin" differently than intended, producing slot machine implementations that deviated significantly from the expected gameplay. Additionally, several candidates failed to deliver the desired functionality altogether, key features either did not work as specified or were missing entirely, regardless of how the refinement prompt was written.



Honest Take for a Manager

AI coding tools are most effective when you have a clear vision of what you want to build, knowing the exact features, structure, and behavior upfront allows you to write precise prompts and evaluate outputs critically. Without that clarity, results are inconsistent and unpredictable.



Final Candidate

candidate-026-refinement-4

Total tokens	99,142
--------------	--------

Lines of code	856
---------------	-----

Files produced	3
----------------	---

Runs in browser	Yes
-----------------	-----

Rubric score	[100 / 100]
--------------	-------------

Why this candidate won:

Consistently outperformed other candidates across all refinement rounds, each iteration showed clear improvement in features, polish, and gameplay while maintaining a stable, working foundation.

Conclusion

Consistency

This experiment demonstrated that AI coding tools can generate functional prototypes quickly, but consistency and reliability remain significant challenges. Across 50 runs with an identical prompt, output quality varied widely in structure, features, and correctness. Refinement prompts helped improve results incrementally, but could not guarantee a fully working product within the 200-word constraint. The most valuable takeaway was that AI tools perform best when the developer has a precise vision, knowing exactly what to build, how to evaluate it, and where the model is likely to fall short. This experiment was less about the quality of the final slot machine and more about learning to work with AI critically and deliberately as a team.